AirWatch India

Viva brief and frontend walkthrough

23AID302 — Big Data Analytics

1. The objective, in one paragraph

Air pollution harms people on a timescale where one hour of warning is useful. The objective of this project is to give people that hour. We learn from thirteen years of Central Pollution Control Board station history, watch live atmospheric data for 230 Indian cities, forecast each city's PM2.5 one hour ahead, and issue a public warning wherever the forecast National AQI crosses 150.

The deliverable is not a chart. It is a message: “Air quality in Jaisalmer is expected to reach AQI 500 (Severe) by 00:00 today, caused by coarse dust. Avoid all outdoor activity.” Everything else — the 154-million-row lakehouse, the model, the streaming engine, Hive — exists to produce that sentence accurately and an hour early.

2. Why this is a Big Data problem, not a spreadsheet problem

A fair question is why any of this needs Hadoop. Four reasons, each a real constraint we hit:

The V	Why it forces distributed tooling here
Volume	154,583,979 rows, 33 GB. Phase 4 alone reads 146M rows and writes 19.7 GB. This does not open in Excel, pandas, or one machine's memory.
Variety	Two different data models: flat fixed-schema CSV and nested self-describing JSON. Spark reads both with native readers and unions them onto one schema in a single job.
Velocity	Alerting is worthless late. Spark Structured Streaming scores micro-batches every 10 seconds at roughly 7 seconds latency.
Veracity	88.9% of the structured rows are synthetic. That has to be proven honest, not assumed — hence five statistical tests plus an independent cross-check.
Value	An hour of warning before pollution arrives, for 230 cities at once.

Concrete proof it needed the cluster: training on a 10-million-row sample of 74.5 million was itself a RAM ceiling on a 15.7 GB machine, and the full-scale feature job had to be tuned for a 4-node cluster before its 19.7 GB write would complete.

3. The frontend: what it is and how it works

One self-contained HTML file: dashboard/AirWatch_India.html. Double-click it — no server, no internet, no cluster needed.

Where its numbers come from

```
Hive → export job → one small JSON → embedded into the page at build time.
```

Every figure on the page came out of a Hive SQL query against the external tables over HDFS Parquet. The page itself does not run those queries — it displays results Hive already computed.

	If the page queried Hive live	What we built
Load time	seconds to minutes (154M rows)	instant
Needs the cluster up?	always	no

Works offline?	no	yes
Survives Docker off?	no	yes

This is the standard serving pattern, not a shortcut: the serving layer precomputes, the page renders. A public warning page that dies when the cluster is down is useless.

Is it live?

Be precise here, because it is the question most likely to catch you out. The pipeline is live — it fetches live CAMS and weather data, enriches on YARN, and scores through the streaming engine. The page shows the

most recent completed cycle, not a feed refreshing every second. Refreshing it is one command: python scripts/run_daily_live_cycle.py --all, which re-fetches, re-enriches, re-scores and rebuilds the page. It is designed to be scheduled daily.

> *Honest phrasing to use: "The data pipeline is live. The page is a published snapshot of the latest run, refreshed by a scheduled job — the same way any production dashboard works."*

4. Inch-by-inch walkthrough of the screen

Top to bottom, what each part is and what to say while pointing at it.

(a) Header — top left

"AirWatch India · Public air-quality warnings · next-hour forecast · CPCB National AQI", with the last-updated timestamp on the right.

> *"This is a public warning board. Everything on it is a one-hour-ahead forecast on India's official CPCB AQI scale, not a foreign standard."*

(b) The red banner — the headline

A large number, the sentence "cities under an air-quality warning", the total warnings issued, and on the right the worst city with its band and cause.

> *"Right now 91 cities are under warning — 1,130 warnings issued across the 230 cities we monitor. The worst is Jaisalmer at AQI 500, Severe, caused by coarse dust. That makes sense: it is a desert city."*

(c) Search box and filter chips

A "Find your city…" box, then chips: All / Moderate / Poor / Very Poor / Severe, and a count on the right.

> *"A resident does not care about 91 cities — they care about theirs. Type a city and you get its warning. The chips filter by severity if you only want the serious ones."*

Demo move: type a city name and let them watch the list filter live. Then clear it.

(d) The warning cards — the actual product

Each card is the message one city's residents receive. Read one out loud; this is the whole project in one object.

Part of the card	What it is
Coloured top edge + band chip	CPCB band colour — Moderate, Poor, Very Poor, Severe
"Warning sent to residents of" + city	Who it is for
The lead sentence	Forecast AQI, the band, the hour it hits, and the cause
Health line	CPCB's own published advisory wording for that band
"What to do"	The action, plus a cause-specific tip
Right now / In 1 hour	Current PM2.5 beside the forecast AQI — the point of the whole system

> *"This is the deliverable. Not a chart — a message someone can act on. The health advice is CPCB's own wording for that band, and the tip changes with the cause: for ozone we say avoid midday exertion; for dust we say wear a mask. Those are different actions, and getting that right is why we track the dominant pollutant per alert."*

(e) The India map

All 230 monitored cities. Colour is the CPCB band, dot size is PM2.5. Hover gives city, state, exact PM2.5, band and timestamp. The worst cities are labelled, with collision handling so names never stack.

> *"Every monitored city, coloured by band and sized by PM2.5. You can see the Punjab–Haryana–Delhi belt light up while the south stays green — the geography of the problem in one glance. Drawn from the live coordinates in the data, no map library."*

(f) Warnings per hour

A bar per forecast hour across the window.

> *"Warnings are not evenly spread — they cluster in specific hours, which is what you would expect if a real physical process is driving them rather than random noise."*

(g) What is causing the warnings — the strongest single finding

Horizontal bars: Ozone 82%, Coarse dust 12%, Fine particles 6%, Sulphur dioxide under 1%.

> *"This is the finding I would point to. 82% of current warnings are driven by ozone, not particulates. That is a September photochemical-smog signal — ozone forms in afternoon sunlight. It is only visible because the alert engine records the dominant pollutant for every alert, and it changes the public advice completely: a mask does nothing for ozone. The right advice is to avoid midday exertion."*

(h) Strongest warnings table

The top 20 by forecast AQI: city, forecast hour, PM2.5 now, PM2.5 in one hour, forecast AQI, band, cause, and the full message verbatim.

> *"These are the raw messages the streaming engine produced — not rewritten for the presentation. The PM2.5-now column beside PM2.5-in-one-hour is the forecast doing its job."*

(i) How these warnings are made

Four tiles: forecast skill (R^2 0.891), cities monitored, history behind the model, and the warning threshold.

> *"And this is the honesty panel — the model's skill is on the page itself, next to its output, not hidden in a report."*

5. The model result

Real rows only (n = 2,074,569)	R^2	RMSE	MAE
Persistence baseline ("next hour = this hour")	0.8587	24.831	9.937
Our GBT forecaster	0.8910	21.806	8.926
Improvement	**+0.0323**	**−12.18%**	**−1.011**

- R^2 0.8910 — the model explains 89.1% of what happens to next-hour PM2.5.
- RMSE 21.8 — a typical miss of about 22 µg/m³.
- MAE 8.9 — the usual miss is under 9; RMSE is higher because rare spikes pull it up.

Why the comparison matters more than the number. Pollution changes slowly, so "assume next hour equals this hour" already scores 0.8587. A model that cannot beat that has learned nothing. We beat it by 12.18% lower error, and our own training script prints *** FAIL *** and refuses the run if we do not.

Setup: 300 trees, depth 6, 241 cities, 5 h 50 m of training. Scored on 18,671,121 test rows, not sampled. We quote the 2,074,569 real-only rows because synthetic rows are easier to predict and a blended figure would flatter us.

6. What the model actually looks at

86 inputs: 54 numeric features plus 32 one-hot columns for season, weather condition and climate type.

Group	Examples
Pollutants now	pm25, pm10, no2, so2, co, o3, nh3, aqi_current
Recent history	pm25_lag_1h, pm25_delta_1h, pm25_roll_3h / 6h, pm25_std_6h, pm25_surge_ratio, pm25_avg24h
Weather	temperature, humidity, wind speed, wind direction (sin/cos), rainfall, pressure, solar radiation
Physics	ventilation_index, scrub_factor, stagnation_index, temp × humidity, is_raining
Time and traffic	hour / month / day-of-week as sin-cos pairs, is_weekend, traffic_index, is_rush_hour
City	urbanization_index, climate_type, season
Provenance	ten *_was_imputed flags — which readings were filled rather than measured

Which ones actually mattered (measured, from the training run)

Feature	Importance	What it means
pm25 (current)	0.101	Where you are now is the strongest clue
pm25_delta_1h	0.076	Whether it is rising or falling
pm25_surge_ratio	0.069	Whether a spike is under way
pm25_lag_1h	0.059	The previous hour
pm25_roll_3h	0.054	The recent trend
pm25_was_imputed	0.054	Whether this reading was measured or filled
hour_sin / hour_cos	0.031 / 0.024	Time of day

The sixth row is worth pointing at. pm25_was_imputed — a flag saying "this value was filled in, not measured" — is the sixth most important input out of 86. The model taught itself to discount filled readings.

That is direct evidence the decision to flag every imputed value, rather than quietly filling them, was the right one.

> *"The model does not just use the numbers — it uses whether it can trust them."*

7. How we proved the dataset is truthful

This is the hardest question we face, because 88.9% of the structured rows are synthetic. Two independent lines of evidence.

(a) Five paired statistical tests — scripts/00d_validate_expansion.py

Every synthetic station is compared against its own real parent station, not against an average. The script's header says: treat FAIL as blocking.

#	Test	The cheat it catches
1	Persistence R^2	THE test. If fake data is easier to predict than real data, every score you report is inflated
2	Autocorrelation, lags 1–6h	Random noise decorrelates; interpolation over-smooths
3	Marginal quantiles p5–p95	Drift and variance inflation
4	Missingness rate	Fake data must be exactly as full of gaps as real data
5	Cross-correlation pm25~pm10, pm25~no2	Per-column noise that destroys relationships between pollutants

Why test 1 is the important one. The previous generator produced synthetic data scoring 0.856 on persistence while real data scored 0.733 — the fake data was far easier to predict, so every headline number was flattered by the synthetic majority. That generator would have failed this test outright. Ours passes.

Why test 4 is the subtle one. The old generator filled the gaps, making synthetic series solid and continuous. Real sensors have holes. Ours copies each real station's gap pattern exactly. The code comment calls densifying "the subtlest way to fake easy data".

(b) An independent outside check

Beyond the generator's own tests, we compared our CPCB data against Copernicus CAMS satellite reanalysis over the eight months where both exist: 1,152,707 matched city-hours.

Pollutant	Correlation	Read
PM2.5	0.565	Moderate — plausible for roadside sensors vs a coarse model grid
PM10	0.558	Moderate
O3	0.361	Weak, with a large systematic offset
NO2	0.219	Weak
SO2	0.083	Essentially no agreement — reported, not hidden

Say this before you are asked: "The cross-validation is a mixed result and we publish it as one. Particulates track moderately. SO2 does not agree at all. We also ran a synthetic-versus-CAMS test and then threw it out, because averaging to city-hour cancels the perturbation — the test could not tell real from synthetic by construction, so it was not evidence either way."

8. Questions you will be asked

Question	Answer
What is the accuracy?	It is regression, so R^2 rather than accuracy: 0.891, and 12.18% better than the baseline on real rows.

Why so much synthetic data?	88.9%, fully disclosed. That is exactly why we report real-rows-only separately — synthetic rows are easier to predict.
Is the dashboard live?	The pipeline is live; the page is a snapshot of the latest completed cycle, refreshed by a scheduled job.
Does the frontend use Hive?	Yes — every number comes from a Hive query. The export job runs the SQL; the page displays the results.
Did the validation pass?	The five paired tests pass. The independent CAMS check is mixed, and we say so: PM2.5 0.565, SO2 0.083.
Why Hadoop for this?	154M rows and a 19.7 GB feature write. A 10M-row training sample was itself a RAM ceiling on one machine.
What went wrong?	Eleven defects, all documented. Two produced plausible wrong answers with no error — 16,560 rows in, 0 out, no exception.

> *If you only remember one line: "We do not just report what worked — every number in this project can be reproduced from a script in the repository, and the weak results are published as plainly as the strong ones."*